HTML QUICK STYLER

Project Report

Submitted to **Manonmaniam Sundaranar** University in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

NAAN MUDHALVAN COURSE
(Agentic AI Application Developer)

Submitted By

M.AMINAL NASRIN	**23081021802112004**
M. JESILA FATHIMA	**23081021802112015**
A.KRISHNAVENI	**23081021802112017**
M.SHAHITHA BANU	**23081021802112029**

Under the guidance of

Mrs. Dr.A. Aaisha Nazleem MCA., M.Phil.,Ph.D

Assistant Professor,

DEPARTMENT OF COMPUTER SCIENCE

ANNAI HAJIRA WOMEN'S COLLEGE

(AFFILIATED TO MANONMANIAM SUNDARANAR UNIVERSITY)

TIRUNELVELI - 627005

MARCH– 2026

CERTIFICATE

This is to certify that the project entitled as **"HTML QUICK STYLER "** is a bonafide work done by _____,
_____, _____,
_____

for the **Naan Mudhalvan Course (Agentic AI Application Development)** in partial fulfillment of the requirements for the award of the degree in Bachelor of Computer Science during the academic year (VI Semester) 2025-2026.

Place:

Date:

Signature of the H.O.D **SPOC**

Submitted for the Naan Mudhalvan university Examination held on _____
at the Department of Computer Science, Annai Hajira Women's College, Tirunelveli.

Principal **External Examiner**

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1.	**ABSTRACT**	
2.	**INTRODUCTION**	1
3.	**SYSTEM REQUIREMENTS** 3.1 HARDWARE REQUIREMENTS 3.2 SOFTWARE REQUIREMENTS 3.3 CORE TECHNOLOGIES	3
4.	**SYSTEM DESIGN** 4.1 TECHNICAL ARCHITECTURE 4.2 PROJECT STRUCTURE	5
5.	**PROJECT WORKFLOW**	7
6.	**ALGORITHM**	8
7.	**IMPLEMENTATION** 7.1 CODING 7.2 OUTPUT	10
8.	**CONCLUSION**	19

HTML QUICK STYLER

1.ABSTRACT

The increasing demand for websites in various fields such as education, business, communication, and information sharing has made web development an important skill in today's digital world. However, creating a website requires knowledge of web technologies such as HTML, CSS, and JavaScript, which can be challenging for beginners and students who are new to programming. Writing structured HTML code manually also requires time, practice, and technical understanding. To simplify this process and make website creation easier, the **HTML Quick Styler** project is developed as an automated web page generation tool. The main objective of this system is to help users quickly create structured and visually styled HTML web pages based on simple user inputs without requiring deep coding knowledge. In this system, the user provides basic information such as the website title, description, images, and image URLs. Based on the given inputs, the application automatically generates properly formatted HTML code that can be used to build a complete webpage. The system also includes several useful features to improve usability and functionality. Users can upload images directly from their local system or provide image URLs to include them in the generated webpage. The application supports multiple image uploads and also provides an image preview option to help users verify the images before generating the final output. In addition, the system includes predefined templates that allow users to apply different layout styles to their webpages. Another important feature of this project is the AI UX Reviewer, which analyzes the generated webpage structure and provides suggestions to improve the user experience and design quality by checking elements such as layout structure, accessibility, and responsiveness. The project is implemented using Python for backend processing and HTML, CSS, and JavaScript for the user interface. Overall, the HTML Quick Styler system provides a simple, efficient, and user-friendly solution for automatically generating web pages while helping beginners understand the basic structure of web development.

2.INTRODUCTION

In the modern digital world, websites play a very important role in communication, information sharing, business promotion, education, and many other activities. Almost every organization and individual requires a website to present information and services online. Web development usually involves designing and writing code using technologies such as HTML, CSS, and JavaScript. However, creating a structured and visually appealing webpage requires technical knowledge and experience. Beginners, students, and non-technical users often find it difficult to understand the coding structure and layout design required to build a proper website. Writing HTML code manually, arranging different sections of a webpage, managing images, and maintaining proper design alignment can be a time-consuming and complex process for new learners.

To overcome these difficulties, the **HTML Quick Styler** project is developed as an automated web page generation tool. The main aim of this system is to simplify the process of creating HTML webpages by generating structured and styled HTML code based on simple user inputs. Instead of writing code manually, users can provide basic content such as website titles, descriptions, images, and image URLs. The system then processes this input and automatically generates a complete HTML webpage with proper formatting and styling. This helps reduce manual coding effort and allows users to focus on the content rather than the technical implementation of the webpage structure.

One of the important features included in this system is **image handling functionality**. The application allows users to upload images directly from their local system as well as provide image URLs. These images can then be integrated into the generated webpage automatically. The system also supports multiple image uploads, which allows users to add several images to different sections of the webpage. In addition, the application provides an **image preview feature** so that users can verify the uploaded images before generating the final HTML output. This improves usability and ensures that the correct images are displayed in the generated webpage.

Another useful component of the project is the **template support feature**. The system provides predefined templates that help users quickly apply structured layouts to their webpages. By selecting a template, the user can instantly generate a webpage design without having to manually create sections such as hero banners, content areas, and footers. This feature helps beginners understand how modern websites are structured and improves the overall design quality of the generated pages.

The system also includes a **live preview feature**, which allows users to immediately view the generated webpage output. Instead of copying the HTML code and opening it in a separate environment, the user can directly see how the webpage will appear in a browser. This real-time visualization helps users understand the relationship between the input content and the final webpage layout.

Another advanced feature included in this project is the **AI UX Reviewer**. This component analyzes the generated HTML webpage and provides suggestions to improve the overall user experience and design quality. It checks important elements such as layout structure, heading hierarchy, accessibility features, and responsiveness. Based on these checks, the system provides feedback and recommendations that help users improve their webpage design according to modern web standards.

The HTML Quick Styler project is implemented using **Python for backend processing**, while **HTML, CSS, and JavaScript are used for the frontend interface**. The system processes user inputs, generates styled HTML code, and displays the final result through an interactive interface. By combining automation, usability, and intelligent feedback features, the project provides an efficient solution for generating webpages quickly and helps students and beginners learn the structure of web development more easily.

3.SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

1. Processor Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz 2.50 GHz
2. Installed RAM 8.00 GB (7.89 GB usable)
3. System type 64-bit operating system, x64-based processor

3.2 SOFTWARE REQUIREMENTS

- **Operating System** : Windows 10
- **Programming Language** : Python
- **Frontend Technologies** : HTML, CSS, JavaScript
- **Framework** : Gradio
- **Development Tool** : Google Colab
- **Browser** : Google Chrome

3.3 CORE TECHNOLOGIES

- **Python**

  Python is the main programming language used in this project. It handles the backend logic, HTML generation, image processing, and overall functionality of the application.

- **Gradio**

  Gradio is used to build the user interface of the system. It allows users to enter page details, upload images, and view the generated HTML output through an interactive web interface.

- **HTML**

  HTML (HyperText Markup Language) is used to create the structure of the generated web pages. It defines different sections such as hero, features, portfolio, contact form, and footer.

- **CSS**

  CSS (Cascading Style Sheets) is used to style the web pages. It controls layout, colors, fonts, responsiveness, and visual design elements of the generated website.

- **Transformers**

  The Transformers library is used to integrate an AI model that helps generate content and assist in creating dynamic web page structures based on user input.

- **Base64 Image Encoding**

  Base64 encoding is used to embed uploaded images directly into the HTML pages. This allows images to be displayed without requiring an external server.

4. SYSTEM DESIGN

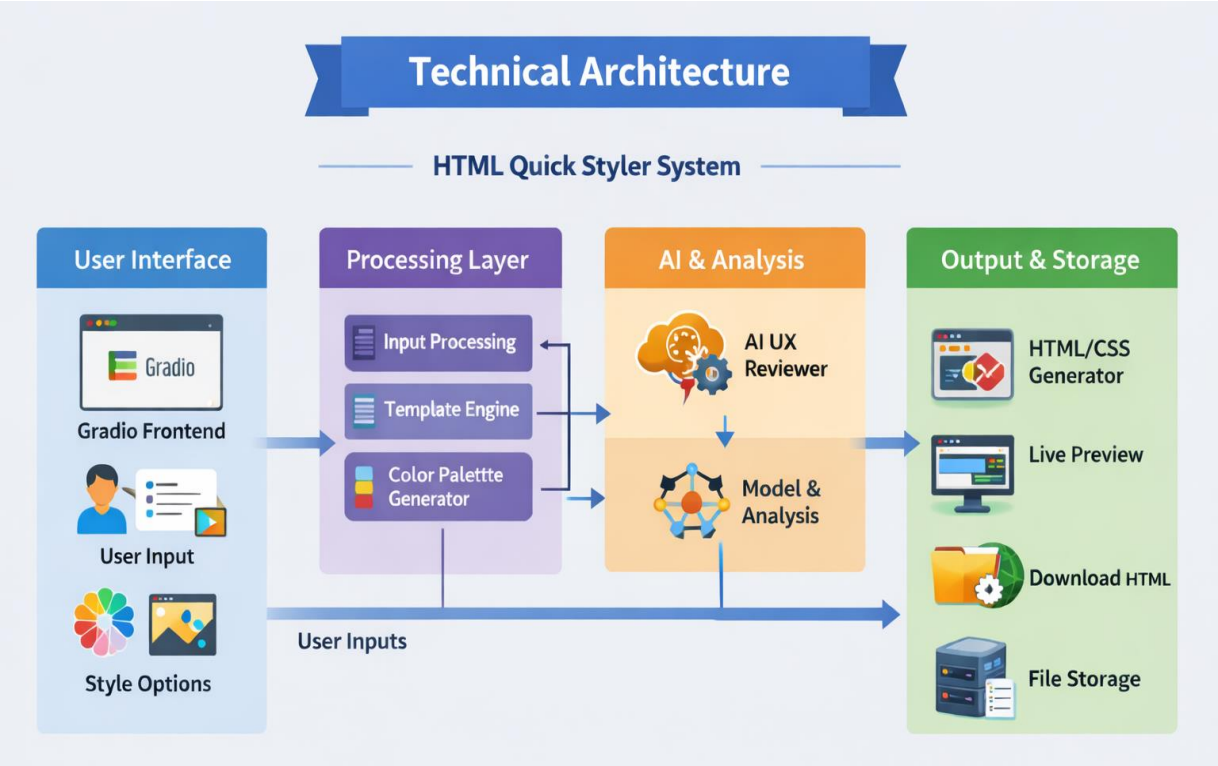
System design explains the structure and organization of the HTML Quick Styler system. It shows how different components of the project work together to generate a complete HTML web page based on user input.

The system is designed with a simple architecture where the user interacts with a web interface to provide inputs such as page title, description, color style, and images. These inputs are processed by the backend system, which generates the required HTML structure and applies styling.

The system mainly consists of three major components: the User Interface, Processing Module, and Output Module. The user interface collects user input, the processing module generates the HTML content and applies styles, and the output module displays the final web page and HTML code.

This design makes it easy for users to create web pages without writing code.

4.1 TECHNICAL ARCHITECTURE



4.2 PROJECT STRUCTURE

The project structure describes how the application is organized to perform the website generation process. Even though the project is simple, it is logically divided into different functional parts inside the main file.

```
HTML Quick Styler Project
│
└── Main Application File
    │
    ├── User Interface Section
    ├── Input Processing Section
    ├── HTML Generation Section
    ├── CSS Styling Section
    └── Output Display Section
```

Description:

- **User Interface Section** – Handles the interaction between the user and the system using Gradio.
- **Input Processing Section** – Processes the data entered by the user such as title, description, and images.
- **HTML Generation Section** – Generates the HTML structure of the web page dynamically.
- **CSS Styling Section** – Applies styles and layouts to make the web page visually attractive.
- **Output Display Section** – Displays the final generated HTML page and preview to the user.

5. PROJECT WORKFLOW

The project workflow explains the step-by-step process of how the HTML Quick Styler system works to generate and customize a web page based on user input.

1. **Generate Page**

   The user enters details such as page title and description. The system processes the input and generates the basic HTML structure of the web page.

2. **Live Preview**

   The generated web page is displayed instantly so the user can see how the page looks in real time.

3. **Color Palette Selection**

   The system provides different color palette options that help users choose suitable colors for the web page design.

4. **AI UX Reviewer**

   This feature analyzes the generated page and provides suggestions to improve the design and user experience.

5. **Template Selection**

   The system also provides predefined templates that users can apply to their web page. These templates include different layouts and design styles that help users quickly create a structured and professional-looking website.

6. **Guide Section**

   The guide section provides instructions and tips that help users understand how to use the system effectively. It assists users in selecting features, applying styles, and generating the final web page easily.

6. ALGORITHM

1. Start & Open Application

• Launch the HTML Quick-Styler application in a web browser.

• Load the main interface with input and output panels.

2. Generate Page

• User enters Page Title and Page Description / Sections.

• User selects style options and uploads images if needed.

• User clicks the **Generate Page** button.

• The system processes the input and generates the HTML structure.

3. Live Preview

• The system displays a live preview of the generated webpage.

• User checks the layout, content, and design of the page.

4. Color Palettes

• The system provides different color palette options.

• User selects a suitable color combination for the webpage design.

5. AI UX Reviewer

• User opens the AI UX Reviewer option.

• The system analyzes the webpage design and provides UX improvement suggestions.

6. Templates

• The system provides different template options.

• User selects a template to apply a predefined layout to the webpage.

7. Download HTML File

• User clicks the **Download HTML** option.

• The system saves the generated webpage as an HTML file on the user's device.

8. Guide

• User can open the Guide section.

• The system displays instructions on how to use the application.

9. End

• User completes the process and exits the application.

7.IMPLEMENTATION

7.1 CODING:

1. Model Initialization

```
from transformers import pipeline
pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")
messages = [{"role": "user", "content": "Who are you?"}]
pipe(messages)
```

2. Color Palette Generator

```
class ColorPaletteGenerator:
    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        palettes = {
            "modern":   {"primary": "#667eea", "secondary": "#764ba2", "accent": "#f093fb",
                        "background": "#0f172a", "surface": "#1e293b",
                        "text_primary": "#f1f5f9", "text_secondary": "#cbd5e1", "border":
"#334155"},
            "vibrant":  {"primary": "#ff6b6b", "secondary": "#ff8e72", "accent": "#f5576c",
                        "background": "#0f172a", "surface": "#1e293b",
                        "text_primary": "#f1f5f9", "text_secondary": "#cbd5e1", "border":
"#334155"},
            "ocean":    {"primary": "#0099cc", "secondary": "#006699", "accent": "#00ccff",
                        "background": "#0f172a", "surface": "#1e293b",
                        "text_primary": "#f1f5f9", "text_secondary": "#cbd5e1", "border":
"#334155"},
        }
        return palettes.get(style.lower(), palettes["modern"])
```

3. Image Provider

```
class ImageProvider:
    @staticmethod
    def picsum(context: str, index: int = 0, width: int = 800, height: int = 500) -> str:
        pool=ImageProvider.IMAGES.get(context, ImageProvider.IMAGES["portfolio"])
```

```
        pic_id = pool[index % len(pool)]
        return f"https://picsum.photos/id/{pic_id}/{width}/{height}"

    @staticmethod
    def file_to_data_url(filepath: str) -> str:
        try:
            mime, _ = mimetypes.guess_type(filepath)
            if not mime or not mime.startswith("image/"):
                mime = "image/jpeg"
            with open(filepath, "rb") as f:
                b64 = base64.b64encode(f.read()).decode("utf-8")
            return f"data:{mime}; base64,{b64}"
        except Exception:
            return ""

    @staticmethod
    def process_uploads(file_list) -> List[Dict]:
        if not file_list:
            return []
        results = []
        for item in file_list:
            filepath = item if isinstance(item, str) else getattr(item, "name", str(item))
            if not filepath or not os.path.isfile(filepath):
                continue
            mime, _ = mimetypes.guess_type(filepath)
            if not mime or not mime.startswith("image/"):
                continue
            data_url = ImageProvider.file_to_data_url(filepath)
            if not data_url:
                continue
            size_kb = round(os.path.getsize(filepath) / 1024, 1)
            results.append({
                "name":     os.path.basename(filepath),
                "path":     filepath,
```

```
            "data_url": data_url,
            "size_kb":  size_kb,
        })
    return results
```

4. HTML Page Builder — Section Keyword Extractor

```
class HTMLPageBuilder:
    @staticmethod
    def extract_sections(description: str) -> List[str]:
        sections = []
        section_keywords = {
            "hero":         ["hero", "banner", "landing"],
            "features":     ["features", "services", "benefits"],
            "about":        ["about", "team", "company"],
            "portfolio":    ["portfolio", "gallery", "work", "projects"],
            "testimonials": ["testimonial", "reviews", "feedback"],
            "pricing":      ["pricing", "plans", "cost"],
            "contact":      ["contact", "form", "reach"],
            "footer":       ["footer", "bottom"],
        }
        desc_lower = description.lower()
        for section_type, keywords in section_keywords.items():
            for kw in keywords:
                if kw in desc_lower:
                    sections.append(section_type)
                    break
        if not sections:
            sections = ["hero", "features", "contact", "footer"]
        return sections
```

5. CSS Generator

```
def generate_css(palette):
    css = f"""
    body {{
        background: {palette['primary']};
        color: white;
        font-family: Arial, sans-serif;
    }}

    header {{
        background: {palette['secondary']};
        padding: 20px;
        text-align: center;
    }}

    section {{
        padding: 40px;
    }}
    """
    return css
```

6. Gradio Interface Launch

```
import gradio as gr

with gr.Blocks() as demo:
    with gr.TabItem("🎯 Generate Page"):
        # Input fields
        title_input    = gr.Textbox(label="Page Title")
        desc_input     = gr.Textbox(label="Page Description & Sections")
        style_dropdown = gr.Dropdown(choices=["modern","vibrant","ocean",
                                              "forest","sunset","luxury",
                                              "creative","minimal"],
                                     label="Color Style", value="modern")
```

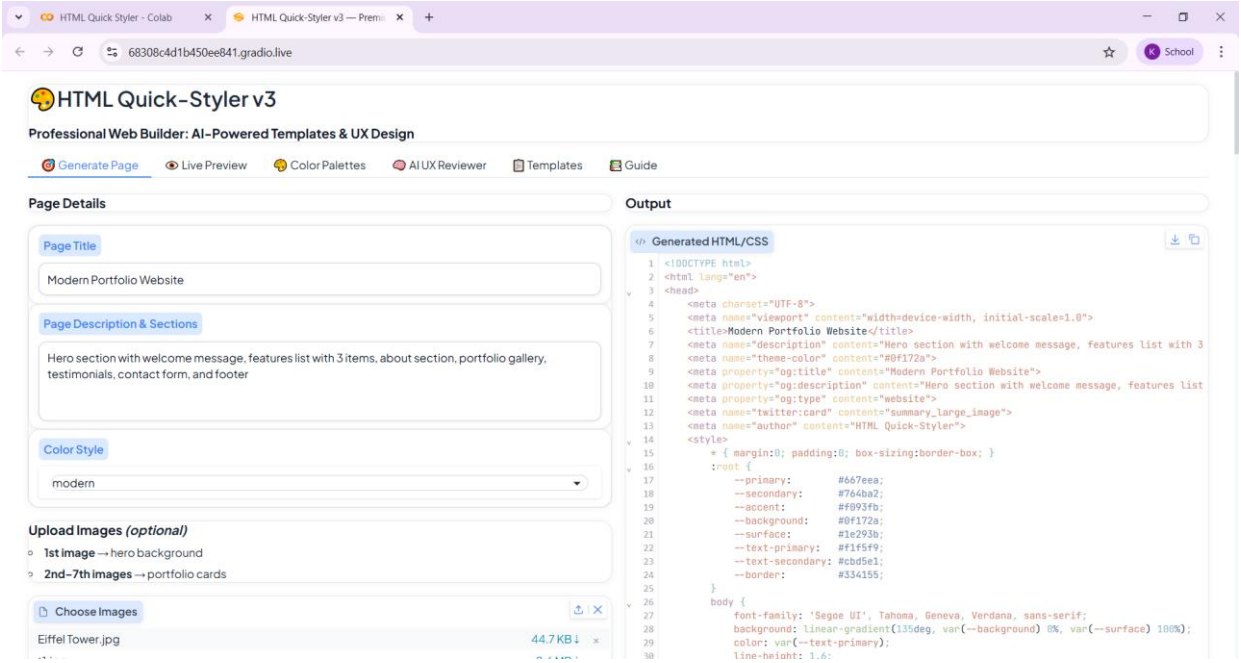
```
        image_upload   = gr.File(label="Upload Images", file_count="multiple")
        generate_btn   = gr.Button("✨ Generate HTML")
        html_output    = gr.Code(label="Generated HTML", language="html")

    with gr.TabItem("👁 Live Preview"):
        preview_btn    = gr.Button("🔄 Load Preview")
        preview_frame  = gr.HTML()

    with gr.TabItem("🤖 AI UX Reviewer"):
        review_btn     = gr.Button("🔍 Run UX Review")
        review_output  = gr.Markdown()

demo.launch(share=True, show_error=True, show_api=False)
```

7. App Launch

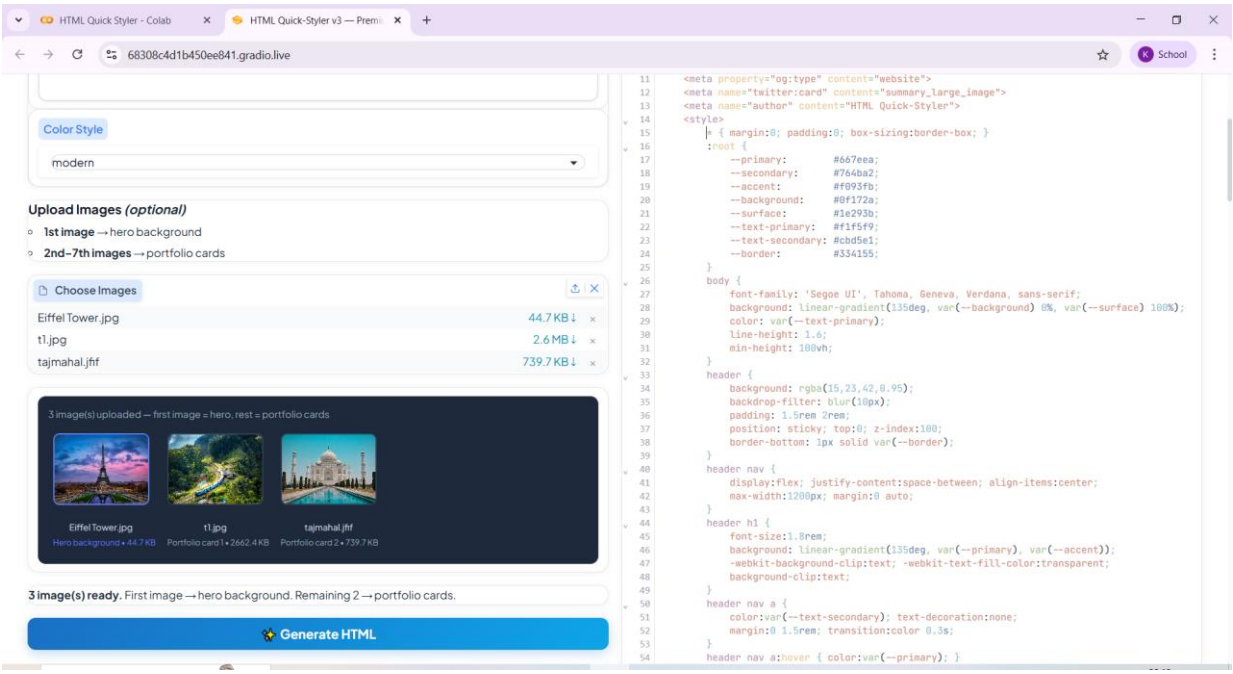
```
print("\n✅ HTML Quick-Styler v3 — Ocean Blue Theme, 9 Templates Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

7.2 OUTPUT:

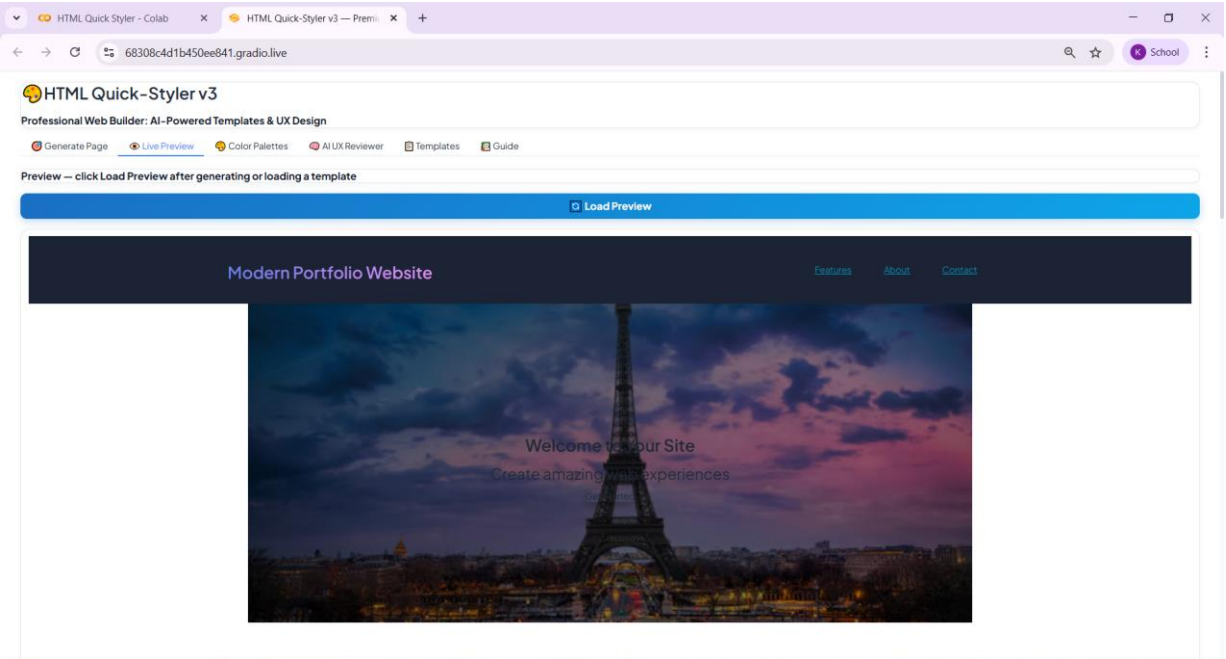
1.HOME PAGE



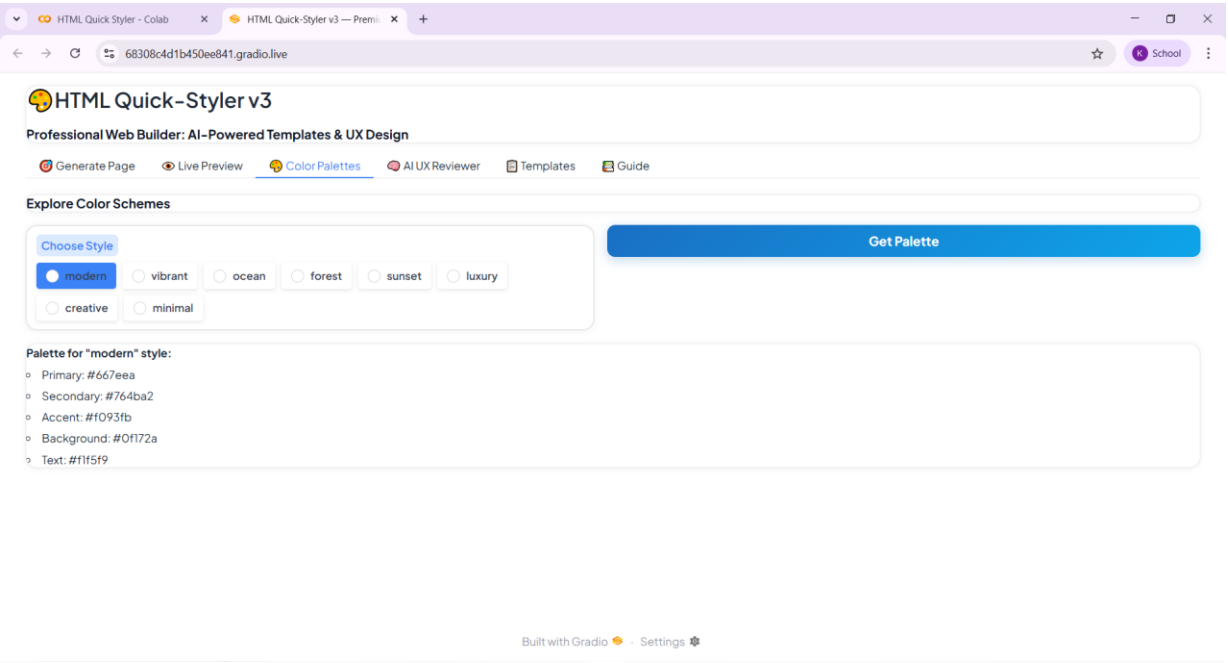
2. IMAGE UPLOAD OPTION



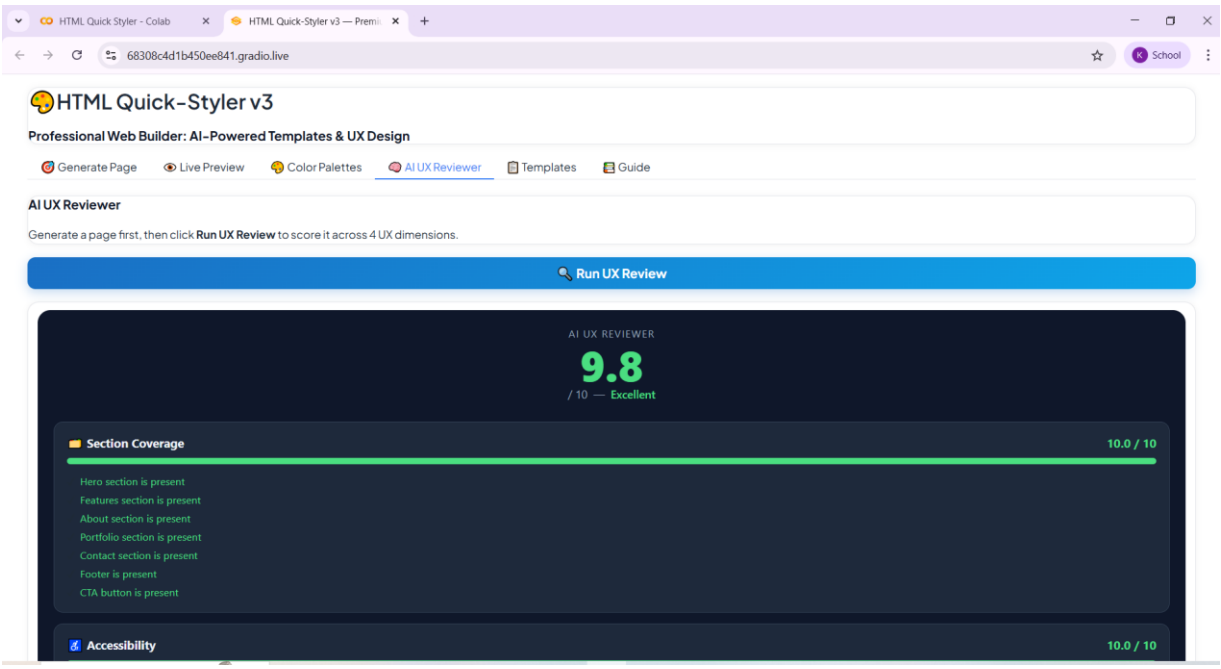
3.LIVE WEBPAGE PREVIEW



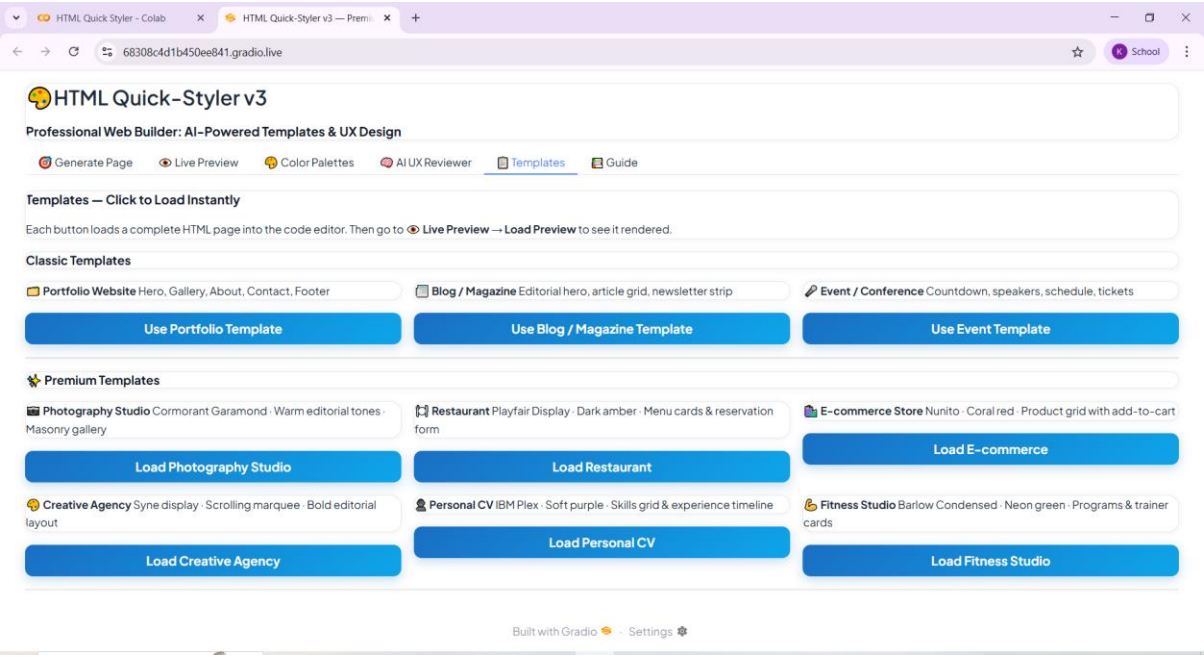
4.COLOR PALETTE PAGE



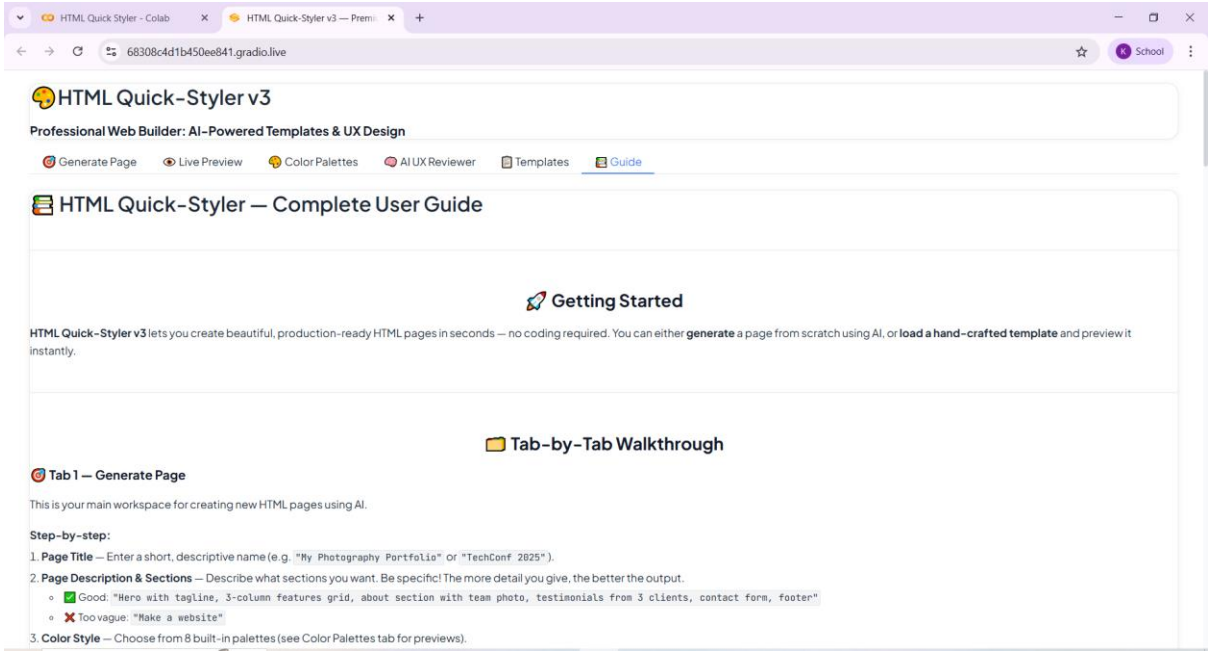
5. AI UX REVIEWER INTERFACE



6. TEMPLATE SELECTION PAGE



7.USER GUIDE PAGE



8.CONCLUSION

The **HTML Quick Styler** project was developed to simplify the process of creating attractive and structured webpages without requiring advanced programming knowledge. In traditional web development, users must manually write HTML and CSS code to design webpages, which can be difficult for beginners and time-consuming for developers. This project addresses that challenge by providing an automated system that generates styled HTML webpages based on simple user inputs.

The application allows users to enter basic webpage information such as the page title, description, and required sections. Users can also select different color palettes and optionally upload images to personalize their webpage. Based on the provided inputs, the system automatically generates the HTML structure and CSS styling required to build the webpage. This makes webpage creation faster, easier, and more accessible for users who may not have strong coding skills.

Another important feature of this project is the user-friendly interface developed using the Gradio framework. The interface allows users to easily interact with the system, generate webpages, and view the generated code. The application also provides a live preview of the webpage so users can immediately see how their webpage design looks. In addition, the AI UX Reviewer analyzes the webpage layout and provides suggestions to improve usability and user experience.

The system also includes useful features such as template selection, color palette options, and the ability to download the generated HTML file. These features make the tool practical for students, beginners, and developers who want to quickly create webpage prototypes.

In conclusion, the HTML Quick Styler project demonstrates how automation and intelligent tools can improve the web development process. By simplifying webpage creation and providing design assistance, the system helps users transform their ideas into functional and visually appealing webpages efficiently.